Practical No. 06

Prediction of Sensor Values using Regression and LSTM Models

Aim

To predict future IoT sensor readings (e.g., temperature) using two approaches — a simple Linear Regression model and a Long Short-Term Memory (LSTM) neural network — and compare their performance on time-series sensor data.

Software / Tools Required

- Google Colab / Jupyter Notebook (Python 3)
- Libraries: numpy, pandas, matplotlib, scikit-learn, tensorflow (Keras)
- No physical hardware required — sensor readings are simulated in software, since real IoT hardware is not available to students.

Theory

1. Why predict sensor values? In IoT systems, sensors continuously stream data (temperature, humidity, vibration, etc.). Predicting the next value(s) is useful for:

- Early warning (e.g., predicting an overheating machine before failure)
- Filling in missing/lost sensor readings
- Reducing how often a sensor needs to transmit data (predict-and-correct instead of always sending)

2. Linear Regression (recap) Linear Regression fits a straight line y = mx + c between an independent variable (time step, or previous readings) and the target (next reading). It assumes a simple, roughly linear relationship and works well for slow-changing trends but struggles with cyclic or highly non-linear sensor patterns (e.g., daily temperature cycles).

3. Time Series Framing To predict a sensor value at time t, we don't just use t as input — we use a window of previous readings (e.g., last 5 values) as features to predict the next one. This is called a "sliding window" approach and converts a raw sequence into a supervised learning problem:

```
Input: [x1, x2, x3, x4, x5] -> Output: x6
Input: [x2, x3, x4, x5, x6] -> Output: x7
...
```

4. What is LSTM, and why use it here? A regular neural network has no memory of past inputs — each prediction is independent. Sensor data, however, is sequential: today's temperature depends on yesterday's trend, daily cycles, seasonal drift, etc.

Recurrent Neural Networks (RNNs) were designed to handle sequences by feeding the output of one time step back in as input to the next, giving the network a form of "memory." But plain RNNs suffer from the vanishing gradient problem — over long sequences, the influence of earlier inputs fades almost to zero, so they struggle to learn long-term patterns.

LSTM (Long Short-Term Memory) networks solve this with a more sophisticated internal structure — a "cell state" that carries information across many time steps, regulated by three gates:

- **Forget gate** — decides what old information to discard
- **Input gate** — decides what new information to add

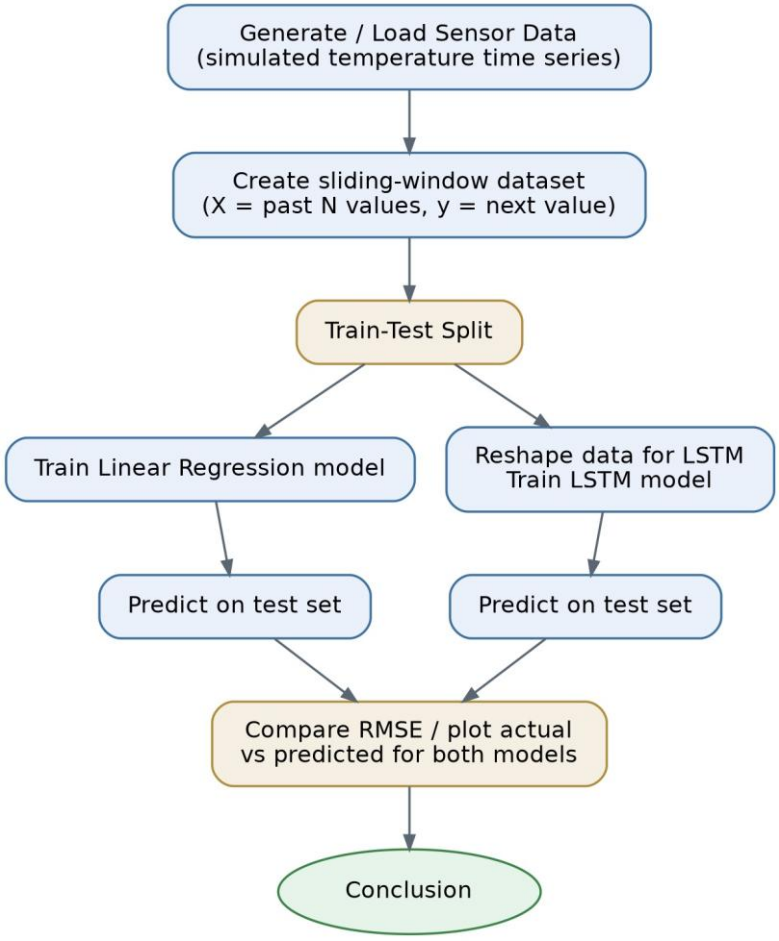
- **Output gate** — decides what to output at this step

This lets LSTMs learn both short-term fluctuations and longer-term trends in sensor data, making them much better suited to time-series prediction than plain regression or vanilla RNNs.

5. Regression vs LSTM — expected comparison

Aspect	Linear Regression	LSTM
Handles non-linearity	Poor	Good
Handles sequential/cyclic patterns	Poor	Good
Training time	Very fast	Slower
Data needed	Small	Larger
Interpretability	High (simple equation)	Low (black box)
Good for	Quick baseline, simple trends	Complex, cyclic sensor behavior

Flowchart



Flowchart for Practical 06

Step-by-Step Procedure

1. Open Google Colab and create a new notebook.
2. Install/import required libraries (numpy, pandas, matplotlib, sklearn, tensorflow).

3. Generate a simulated temperature sensor dataset: a sine wave (daily cycle) plus random noise, over a few hundred time steps — this mimics real sensor behavior without needing hardware.
4. Convert the 1-D time series into a supervised sliding-window dataset (input = last 5 readings, output = next reading).
5. Split the data into training and testing sets (e.g., 80/20).
6. Train a **Linear Regression** model on the windowed data.
7. Reshape the same windowed data into 3-D shape (samples, timesteps, features) required by LSTM, and build a small **LSTM model** (1 LSTM layer + 1 Dense output layer).
8. Train the LSTM model for a small number of epochs (10–20 is enough for this dataset size).
9. Use both trained models to predict on the test set.
10. Calculate RMSE (Root Mean Squared Error) for both models and plot actual vs predicted curves for visual comparison.
11. Record observations comparing the two models' accuracy and behavior.

Program (Python — run in Google Colab)

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# 1. Simulate sensor data (temperature with daily cycle + noise)
np.random.seed(42)
t = np.arange(0, 500)
temperature = 25 + 5*np.sin(2*np.pi*t/50) + np.random.normal(0, 0.5, size=len(t))

# 2. Create sliding-window dataset
def create_windows(data, window_size=5):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i+window_size])
        y.append(data[i+window_size])
    return np.array(X), np.array(y)

window_size = 5
X, y = create_windows(temperature, window_size)

# 3. Train-test split (no shuffling - preserve time order)
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# 4. Linear Regression
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
print("Linear Regression RMSE:", lr_rmse)

# 5. LSTM - reshape to (samples, timesteps, features)
X_train_lstm = X_train.reshape((X_train.shape[0], X_train.shape[1], 1))
X_test_lstm = X_test.reshape((X_test.shape[0], X_test.shape[1], 1))

lstm_model = Sequential([
    LSTM(32, activation='tanh', input_shape=(window_size, 1)),
    Dense(1)
])
```

```
lstm_model.compile(optimizer='adam', loss='mse')
lstm_model.fit(X_train_lstm, y_train, epochs=20, batch_size=8, verbose=0)
lstm_pred = lstm_model.predict(X_test_lstm).flatten()
lstm_rmse = np.sqrt(mean_squared_error(y_test, lstm_pred))
print("LSTM RMSE:", lstm_rmse)

# 6. Plot comparison
plt.figure(figsize=(10,5))
plt.plot(y_test, label='Actual')
plt.plot(lr_pred, label='Linear Regression Prediction')
plt.plot(lstm_pred, label='LSTM Prediction')
plt.legend()
plt.title('Actual vs Predicted Sensor Values')
plt.xlabel('Time step')
plt.ylabel('Temperature')
plt.show()
```

Expected Output

- Two RMSE values printed (LSTM should generally be lower/better since the data has a cyclic pattern that regression can't capture well).
- A plot showing three curves: actual sensor readings, regression predictions (likely a smoother, less accurate fit), and LSTM predictions (should track the cyclic pattern more closely).

Conclusion

Linear Regression provides a fast, simple baseline for sensor value prediction but fails to capture cyclic/non-linear patterns in the data (such as the daily temperature cycle simulated here). The LSTM model, owing to its ability to retain information across time steps through its gated memory cell, models the periodic behavior of the sensor data more accurately, resulting in a lower RMSE. This demonstrates why sequence-aware models like LSTM are generally preferred for real-world IoT sensor prediction tasks, despite requiring more computation and data than simple regression.

Exercise

Modify the simulated dataset to add a sudden anomaly (a sharp spike) in the temperature readings partway through the series. Retrain both models and observe how each one responds to the anomaly in its predictions.

Exercise Solution

Approach

The same simulated temperature series was used (sine wave, period 50, plus Gaussian noise, 500 samples, seed 42). A single-point anomaly of **+15 °C** was added at **t = 420**, which lies inside the 20% test portion (the test set covers t = 399 to 494). Both models were then retrained on the modified series with the same settings as the practical (window = 5, 80/20 chronological split; LSTM: 32 units, Dense(1), Adam, MSE, batch size 8, 20 epochs). The baseline (no anomaly) was run alongside for comparison.

Important modification: inputs to the LSTM were standardised using the mean and standard deviation of the training windows only (and predictions converted back to °C). Without scaling, raw values of about 25 °C saturate the LSTM gates and the network, in 20 epochs, collapsed to a constant output (RMSE 3.68 on clean data). Scaling is standard practice for neural networks and does not affect Linear Regression.

Program (modified parts marked with comments)

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# 1. Simulate sensor data (temperature with daily cycle + noise)
np.random.seed(42)
t = np.arange(0, 500)
temperature = 25 + 5*np.sin(2*np.pi*t/50) + np.random.normal(0, 0.5, size=len(t))

# >>> EXERCISE: inject a sudden anomaly (sharp spike) <<<
ANOMALY_T = 420
temperature[ANOMALY_T] += 15          # +15 deg C spike

# 2. Sliding-window dataset
def create_windows(data, window_size=5):
    X, y = [], []
    for i in range(len(data) - window_size):
        X.append(data[i:i+window_size]); y.append(data[i+window_size])
    return np.array(X), np.array(y)

window_size = 5
X, y = create_windows(temperature, window_size)

# 3. Chronological 80/20 split
split = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# 4. Linear Regression
lr_model = LinearRegression().fit(X_train, y_train)
lr_pred = lr_model.predict(X_test)
lr_rmse = np.sqrt(mean_squared_error(y_test, lr_pred))
print("Linear Regression RMSE:", lr_rmse)

# 5. LSTM  (>>> inputs standardised with TRAIN statistics <<<)
mu, sd = X_train.mean(), X_train.std()
X_train_lstm = ((X_train - mu) / sd).reshape((-1, window_size, 1))
X_test_lstm  = ((X_test  - mu) / sd).reshape((-1, window_size, 1))

lstm_model = Sequential([
    LSTM(32, activation='tanh', input_shape=(window_size, 1)),
    Dense(1)
])
lstm_model.compile(optimizer='adam', loss='mse')
```

```
lstm_model.fit(X_train_lstm, (y_train - mu) / sd, epochs=20, batch_size=8, verbose=0)
lstm_pred = lstm_model.predict(X_test_lstm).flatten() * sd + mu   # back to deg C
lstm_rmse = np.sqrt(mean_squared_error(y_test, lstm_pred))
print("LSTM RMSE:", lstm_rmse)

# 6. Plot
x_axis = np.arange(split + window_size, split + window_size + len(y_test))
plt.figure(figsize=(10,5))
plt.plot(x_axis, y_test, 'k', label='Actual')
plt.plot(x_axis, lr_pred, label='Linear Regression Prediction')
plt.plot(x_axis, lstm_pred, label='LSTM Prediction')
plt.axvline(ANOMALY_T, color='r', linestyle='--', label='Anomaly')
plt.legend(); plt.title('Actual vs Predicted Sensor Values (with anomaly)')
plt.xlabel('Time step'); plt.ylabel('Temperature (deg C)'); plt.show()
```

Output

Note: the results below were generated with an equivalent NumPy implementation of the same LSTM architecture and hyper-parameters (TensorFlow was not available in the environment used). Re-running the Keras program above in Colab will give very similar, but not bit-identical, values because of different weight initialisation.

```
Baseline (no anomaly)
Linear Regression RMSE: 0.7251
LSTM RMSE:              0.7153

With anomaly (+15 deg C spike at t=420)
Linear Regression RMSE: 2.2822
LSTM RMSE:              1.9168
```

Table 1: RMSE comparison

Case	Linear Regression	LSTM
Baseline (no anomaly)	0.7251	0.7153
With spike at t = 420	2.2822	1.9168
With spike, excluding 6 spike-affected steps	0.7313	0.7192

Table 2: Predictions around the anomaly (°C)

t	Actual	Lin. Reg.	LSTM	LR error	LSTM error
419	28.46	29.11	29.17	+0.65	+0.71
420	43.97	28.34	28.60	-15.63	-15.37
421	28.29	38.38	34.05	+10.10	+5.76
422	26.72	35.62	32.40	+8.90	+5.69
423	26.73	30.00	28.87	+3.27	+2.15
424	25.95	23.45	25.83	-2.50	-0.12
425	25.68	20.83	22.64	-4.85	-3.04
426	23.89	25.07	24.92	+1.18	+1.03
427	24.10	23.99	24.02	-0.11	-0.08

Shift in predictions for t = 421 to 425 (windows containing the spike) relative to the no-anomaly run: Linear Regression mean 5.60 °C (max 10.25); LSTM mean 2.99 °C (max 5.79).

Plot

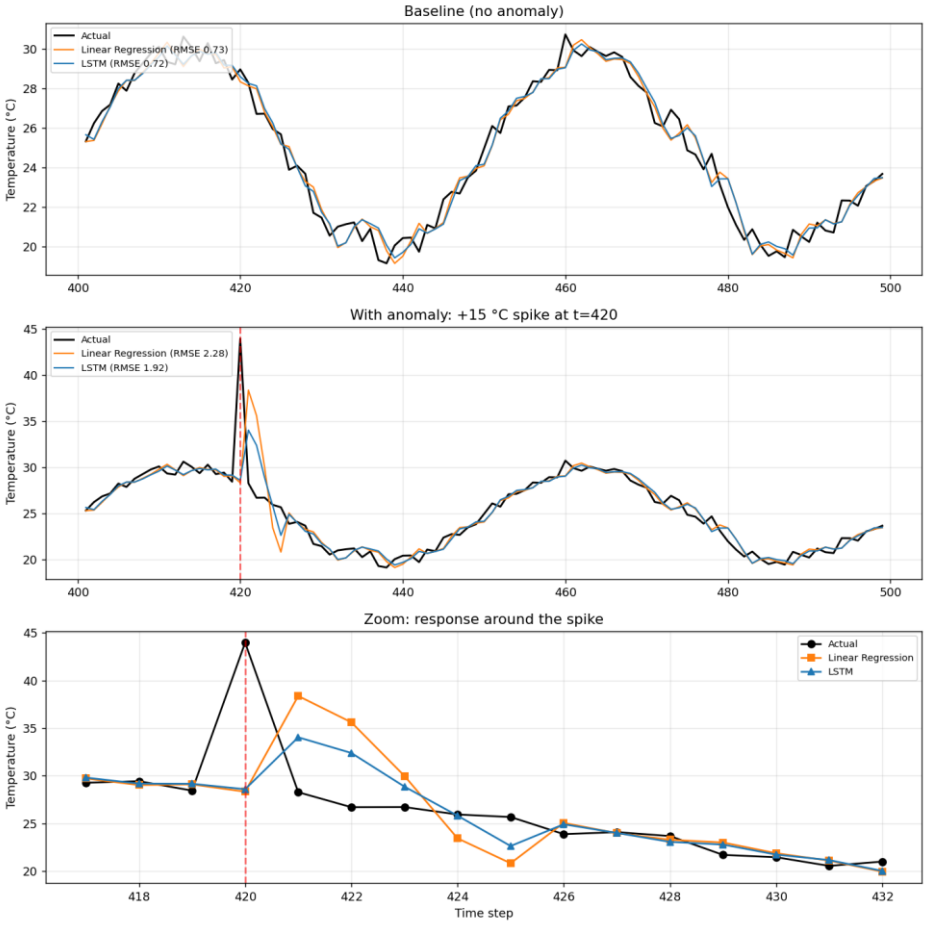


Figure 1: Top – baseline; middle – with +15 °C spike at t = 420 (red dashed line); bottom – zoom on the response around the spike.

Observations

- **Neither model can predict the spike itself.** A sudden spike is not predictable from past values, so both models are off by about 15 °C at t = 420 (errors −15.63 and −15.37).
- **The spike contaminates the next five predictions.** Because the model input is a sliding window of the last 5 readings, the spike stays inside the input window for t = 421 to 425, so those predictions are also wrong even though the actual sensor readings have returned to normal.
- **Linear Regression reacts more strongly.** It applies fixed weights to the window, so the outlier is passed straight through: it overshoots to 38.38 at t = 421 (error +10.10) and then undershoots to 20.83 at t = 425 (error −4.85). Its predictions shifted by 5.60 °C on average (max 10.25).
- **The LSTM is affected less and recovers more smoothly.** Its gated memory dampens the outlier: overshoot of 34.05 at t = 421 (error +5.76) and a smaller undershoot at t = 425 (error −3.04). Mean prediction shift was 2.99 °C (max 5.79).
- **Both models recover completely once the spike leaves the window** (from t = 426 onwards). Excluding the six spike-affected steps, RMSE is back to about 0.72–0.73 for both, i.e. the noise floor of the data (sigma = 0.5). The increase in overall RMSE is therefore caused entirely by the anomaly.
- **Overall effect:** the spike increased the RMSE of Linear Regression from 0.73 to 2.28 and that of the LSTM from 0.72 to 1.92.

- **On clean data the two models are almost equal** (0.725 vs 0.715). A sine wave obeys a linear recurrence, so a 5-point linear model already fits it up to the noise; the LSTM's advantage in this experiment therefore appears mainly in robustness to the anomaly rather than in clean-data accuracy.

Conclusion of the Exercise

Adding a sharp spike to the sensor series increased the prediction error of both models, because a sudden anomaly cannot be forecast and, through the sliding window, also corrupts the following five predictions. Linear Regression amplified the outlier (overshoot then undershoot), whereas the LSTM's gated memory cell dampened it, giving smaller errors (RMSE 1.92 vs 2.28) and a smoother recovery. Both models returned to normal accuracy as soon as the spike left the input window. In a real IoT system, such prediction errors could be used for anomaly detection (a large gap between predicted and actual value flags a faulty or abnormal reading), while the models themselves would need robust pre-processing (outlier filtering) so that a single bad reading does not distort subsequent forecasts. Inputs must also be scaled for the LSTM to train properly.